# Skill Engineering




从一次真实任务开始，把成功的方法沉淀成 Skill。看课件理解目标，再到“动手做”独立完成普通 Chat、提炼、安装和新 Chat 验证。完成基础检查后，可以结束本课；深入阅读解释原理和边界，进阶跟练按需继续。

## 学习顺序

普通 Chat 完成任务 → 提炼 Skill → 安装 → 新 Chat 验证

看课件了解目标，到“动手做”完成基础四步。深入阅读讲原理，进阶跟练选做；PPT 备注和本手册提供逐页讲解。

# 为什么需要 Skill Engineering




## 本页讲解

比如，今天花了几轮对话把日报做好，明天换个 Chat，同样的日期、去重和平均耗时又得重新解释。我们可以把今天用过的数据、确认的规则和核对过的结果留下来，再请 AI 把处理方法整理成 Skill。以后拿到同类数据，就有一套现成的方法可以继续用。交给同事时，他也能顺着这些资料看懂这份日报是怎么算出来、怎么验过的。

**深入理解本页原理**

# 从一次数据处理任务说起




## 本页讲解

这 7 行就是我们交给 AI 的原始明细。T004 出现了两次，同一张工单只能算一次；T006 是 10 月 11 日创建的，也不该进入 10 日的日报。处理后剩下 5 张工单，其中 3 张已完成、2 张待处理。完成的 3 张分别用了 60、45、90 分钟，加起来 195 分钟，除以 3，平均就是 65 分钟。我们希望 AI 把明细整理成这样的日报，并让人能核对它怎么算的。把这些规则保存成 Skill，下次拿到新数据，就有了可以沿用的方法。

**打开第一步完整操作**

# Chat、Skill、Skill Engineering




## 本页讲解

刚才那份日报，在一个 Chat 里做对，就完成了当次任务。把规则和处理脚本保存成 Skill，下一次便有了可调用的方法。不过，另一位同事拿到新数据，也能做对吗？还要实际试一试。我们讲的 Skill Engineering，就是在沉淀方法之后，继续检查换对话、换数据、遇到异常时会发生什么。这样才知道，保存的方法离开原来的聊天还能不能工作。

**深入理解本页原理**

# 哪些工作适合做成 Skill？




## 本页讲解

选择第一项实践时，先找经常重复、方法能说明、成果能核对的工作，例如周报整理、明细核对或按模板准备材料。再确认资料在哪里，AI 能访问和操作什么，允许读取、修改或发送的范围是什么。AI 能自主执行和纠错，仍可能误解要求或漏掉问题；数字要核对明细，分析要核对来源，数据冲突时应停下来请人确认。先在小范围试用，保留结果与核对记录，观察表现是否稳定。评估长期价值时，把节省的时间、复核、修错、费用和维护一起算。具体工具接入方式在深入阅读的补充说明中介绍。

深入理解本页原理

# 一个数据处理 Skill 的两个部分




## 本页讲解

算出 65 分钟之前，还有不少判断要做。统计哪一天、字段齐不齐、重复工单能不能合并，都影响最后的结果。假如同一个 ID 的两行耗时不同，就要停下来弄清原因，不能悄悄选一个。图里的 Workflow 管这些推进和停止的决定，Data Processing 按确认的规则去重、筛选和计算。两部分接起来，结果验过了，才把日报交出去。

**深入理解本页原理**

# AI 推理 vs 确定性执行




## 本页讲解

拿刚才的平均耗时来说，195 除以 3，适合交给固定脚本，下一次也按同一规则计算。AI 则可以帮助理解需求：客户说的日期是创建日期，还是完成日期？字段名称换了，含义是否相同？把这些问题说清楚之后，再交给工具执行。脚本也可能写错分母，所以我们还要拿已知结果的样例核对，不能因为用了代码就省掉验证。

深入理解本页原理

# 一个好的 Skill 长什么样




## 本页讲解

参考包的入口 SKILL.md 说明何时使用、怎样完成和停止；业务规则在 references，处理代码在 scripts/pipeline.py。tests 保存程序检查、输入样例和已确认答案；evals/evals.json 保存让 Agent 完成的任务与核对项，直接引用 tests 的样例和答案。它可以由人或负责评审的 LLM 读取。任务实际执行后，再把输出和工具执行记录交给评审方；仅有配置文件不会产生评估结果。当前参考包没有自动评估程序，tests/check-regression.py 只检查固定脚本。简单任务仍可只有 SKILL.md，模板等资源按需放 assets。

深入理解本页原理

# Qoder 实操：沉淀一个 Skill




## 本页讲解

先在普通 Chat 中做对日报，再把规则和实际使用的脚本整理成 Skill ZIP，安装后开新 Chat。第一次还没保存规则，需要把业务口径说明白；安装后只需说“用 normal.json 生成 2026-10-10 的工单日报，保存到新的结果文件夹”。每次都重讲算法，就无法看出 Skill 是否真的保存了方法。先用 / 选择技能检查执行，再不选择技能，用自然语言试它能否自动找到。

**开始四步跟练**

# 如何验证并改进 Skill




## 本页讲解

工单跟练先验证结果与安装复用。更完整的 Task Evals 使用固定的真实任务和相同 Tests 比较版本：10 个 Golden Tasks 覆盖登录、支付、权限、超时和异常处理；假设 V1 完成 6 个、V2 完成 8 个，成功率从 60% 到 80%，提升 20 个百分点，同时比较质量、耗时、Token 和失败原因。数字仅为示例，需重复运行并控制模型、工具、数据和环境。评分可用程序、人工或 LLM。例如，参考包正常任务完成后，另开评审 Chat，提供 evals.json 的场景 1、已确认答案、实际日报及工具执行记录，请它逐项给出结论和证据。日报为 65，但没有实际调用日志时，调用项应为 UNKNOWN；日报为 39 时，数字项为 FAIL。只让 LLM 核对已有材料不会重跑任务，当前没有自动评估程序。Trigger 看正确触发，Production 持续观察真实质量、成本与失败。

**深入理解本页原理**

# 基础完成后的可选进阶




## 本页讲解

先在“动手做”完成四步与末尾检查，再决定是否继续。想完善自己的 Skill 时，打开选做的进阶跟练：先做只读自评或检查计划，确认标准，再生成并运行 tests/evals。程序检查和 Agent 任务分别执行、保留记录，不把生成文件当作通过。想理解为什么这样验证，可以选读 Tests、Evals 与版本比较原理。基础四步完成就已达成本课练习目标。

选做：打开进阶跟练